

ANALISIS KOMPLEKSITAS ASIMTOTIK DAN EFISIENSI PARADIGMA DIVIDE AND CONQUER PADA ALGORITMA BINARY SEARCH

Topik Pembahasan: Analisis Kompleksitas Algoritma (Materi Pertemuan 2)

Kelompok 9:

SYAMSUL ARIFIN¹, MOH. OFIKURRAHMAN², SHOFYAN IBNU GAZALI³

1. PENDAHULUAN

A. Latar Belakang

Dalam era komputasi modern, volume data yang diproses oleh berbagai perangkat lunak terus bertumbuh secara eksponensial. Hal ini menuntut sistem untuk tidak hanya menghasilkan keluaran yang benar secara fungsional, tetapi juga mampu mengeksekusi instruksi dalam waktu yang optimal. Efisiensi sistem saat ini tidak bisa lagi hanya bergantung pada spesifikasi perangkat keras (*hardware*), melainkan sangat dipengaruhi oleh desain algoritma dan struktur data yang digunakan.

Salah satu operasi paling mendasar yang menjadi fondasi berbagai sistem informasi adalah operasi pencarian data (*searching*) (Widodoputro, Harsani, & Mulyana, 2018). Jika sebuah algoritma pencarian tidak dirancang dengan efisien, penambahan jumlah data yang kecil sekalipun dapat memicu penurunan performa sistem secara drastis (*bottleneck*) (Religia, Rohim, & Hapsari, 2019). Laporan Ujian Akhir Semester (UAS) ini disusun dengan mengangkat spesifik topik dasar mengenai Analisis Kompleksitas Algoritma. Fokus utama dalam laporan ini adalah membedah secara radikal efisiensi dari algoritma Binary Search (Pencarian Biner) yang berlandaskan paradigma *Divide and Conquer*, serta membuktikan keunggulannya dibandingkan metode pencarian linear konvensional secara matematis (Ramadhan, Fauziah, & Lantana, 2023).

B. Identifikasi Masalah

Berdasarkan latar belakang tersebut, beberapa permasalahan utama yang dapat diidentifikasi adalah:

- 1) Kurangnya pemahaman mengenai cara mengukur efisiensi suatu algoritma secara objektif tanpa terdistraksi oleh kecepatan prosesor komputer.
- 2) Pendekatan pencarian sekuensial (*Linear Search*) sering kali tidak relevan lagi untuk menangani himpunan data berskala masif.
- 3) Kebutuhan akan pembuktian sistematis mengenai bagaimana algoritma *Binary Search* mampu mereduksi waktu komputasi secara logaritmik.

C. Rumusan Masalah

Agar pembahasan dalam laporan ini terarah dan mendalam, dirumuskan beberapa pertanyaan penelitian sebagai berikut:

- 1) Bagaimana penelusuran (*tracing*) batas ruang pencarian berjalan saat algoritma *Binary Search* mengevaluasi suatu array yang telah terurut?

- 2) Bagaimana formulasi pembuktian matematis yang menunjukkan bahwa kompleksitas waktu kasus terburuk (*worst-case*) dari *Binary Search* adalah $O(\log n)$?
- 3) Apa implikasi alokasi memori internal *Binary Search* terhadap analisis kompleksitas ruangnya?

D. Tujuan Penulisan

Tujuan dari penyusunan laporan akademis ini adalah:

- 1) Memenuhi syarat kelulusan evaluasi Ujian Akhir Semester (UAS) pada mata kuliah Analisis Algoritma.
- 2) Mendemonstrasikan pemahaman teoretis mengenai evaluasi kinerja algoritma menggunakan Notasi Asimtotik.
- 3) Menyajikan dokumentasi sistematis mulai dari penelusuran manual, analisis matematis, hingga implementasi riil dalam kode pemrograman.

2. DASAR TEORI

A. Analisis Kompleksitas Asimtotik

Analisis asimtotik adalah pendekatan matematis yang digunakan untuk mengevaluasi laju pertumbuhan fungsi (*growth rate*) waktu atau memori sebuah algoritma seiring dengan membesarnya ukuran data input (n). Terdapat tiga notasi utama yang sering digunakan:

- Big-O (O): Menunjukkan batas atas (*upper bound*) dari waktu eksekusi algoritma. Ini merepresentasikan performa pada kasus terburuk (*worst-case scenario*).
- Big-Omega (Ω): Menunjukkan batas bawah (*lower bound*) atau kasus terbaik (*best-case scenario*).
- Big-Theta (Θ): Menunjukkan batas ketat (*tight bound*) untuk kasus rata-rata (Cormen, Leiserson, Rivest, & Stein, 2022).

B. Paradigma Divide and Conquer

Divide and Conquer adalah paradigma perancangan algoritma yang bekerja dengan memecah sebuah masalah besar menjadi sub-masalah yang lebih kecil (*Divide*), menyelesaikan sub-masalah tersebut secara independen (*Conquer*), lalu menggabungkan hasilnya jika diperlukan. Pada metode pencarian, paradigma ini digunakan untuk mengeleminasi sebagian besar elemen data yang tidak relevan di awal proses.

C. Algoritma Binary Search

Algoritma ini mensyaratkan satu aturan mutlak: data harus dalam kondisi terurut (*sorted*). Algoritma bekerja dengan menentukan titik tengah dari himpunan data, lalu membandingkannya dengan nilai target. Jika nilai target lebih besar dari titik tengah, maka seluruh data di paruh kiri otomatis diabaikan, dan algoritma hanya akan melanjutkan pencarian di paruh kanan (Putri & Wibisono, 2022). Proses pembagian dua ini terus berulang hingga target ditemukan (Haming, Lestanti, & Budiman, 2021).

3. METODOLOGI DAN STUDI KASUS

A. Spesifikasi Kasus dan Data Uji

Untuk memberikan gambaran yang konkret, laporan ini menggunakan sebuah studi kasus representatif dengan dataset berskala kecil namun mencerminkan pola komputasi secara utuh.

- Karakteristik Data: Array tipe integer, diurutkan secara *Ascending* (menaik).
- Ukuran Input (n): 10 Elemen (Indeks 0 hingga 9).
- Nilai Elemen: [5, 12, 18, 23, 35, 44, 57, 62, 78, 89]
- Nilai Target yang Dicari: 57

B. Kondisi Batas dan Variabel Kontrol

Sistem menggunakan tiga variabel penunjuk (pointer) utama:

- *low*: Batas bawah indeks rentang pencarian aktif.
- *high*: Batas atas indeks rentang pencarian aktif.
- *mid*: Titik tengah rentang pencarian aktif, diformulasikan dengan $\lfloor (low + high)/2 \rfloor$.

4. PEMBAHASAN DAN TRACING SISTEMATIS

Berikut adalah visualisasi matriks untuk penelusuran batas ruang pencarian pada setiap iterasi hingga nilai 57 berhasil ditemukan:

Tabel 1. Penelusuran (Tracing) Batas Ruang Pencarian Algoritma Binary Search

Nilai low	Nilai high	Formula Indeks Tengah (mid)	Nilai Array[mid]	Evaluasi Logika & Pergeseran Pointer
0	9	$\lfloor (0 + 9)/2 \rfloor = 4$	35	Target (57) > 35. Angka berada di paruh kanan. Batas kiri bergeser: $low = mid + 1 \rightarrow 5$.
5	9	$\lfloor (5 + 9)/2 \rfloor = 7$	62	Target (57) < 62. Angka berada di paruh kiri. Batas kanan bergeser: $high = mid - 1 \rightarrow 6$.
5	6	$\lfloor (5 + 6)/2 \rfloor = 5$	44	Target (57) > 44. Angka berada di paruh kanan. Batas kiri bergeser: $low = mid + 1 \rightarrow 6$.
6	6	$\lfloor (6 + 6)/2 \rfloor = 6$	57	Nilai Array[6] (57) == Target (57). Target Ditemukan!

Dalam skenario pencarian konvensional (Linear), mencari angka 57 di posisi indeks ke-6 membutuhkan 7 kali pengecekan. Namun, dengan memanfaatkan pembagian rentang *Binary Search*, target yang sama bisa dikunci secara presisi hanya dalam 4 kali iterasi evaluasi (Aviantika, Kustanto, & Hasbi, 2021).

5. ANALISIS MATEMATIS ASIMTOTIK

A. Pembuktian Kompleksitas Waktu (*Time Complexity*)

Untuk membuktikan notasi Big-O secara rigid, kita merumuskan penyusutan himpunan data. Misalkan n adalah jumlah elemen awal. Pada setiap langkah komputasi, algoritma membuang setengah bagian array, sehingga sisa data adalah:

- Tahap Awal: n
- Tahap ke-1: $\frac{n}{2^1}$
- Tahap ke-2: $\frac{n}{2^2}$
- Tahap ke- k : $\frac{n}{2^k}$

Kasus terburuk (*worst-case*) terjadi ketika target tidak ada atau terletak di ujung percabangan, di mana rentang pencarian menyusut hingga tepat menyisakan 1 elemen akhir. Persamaan ekuivalensinya adalah:

$$\frac{n}{2^k} = 1$$

Melakukan perkalian silang:

$$n = 2^k$$

Untuk mengisolasi nilai k (jumlah iterasi maksimal yang dibutuhkan sistem), kita mengaplikasikan operasi logaritma basis 2 ($\log_2$) pada kedua ruas persamaan:

$$\log_2(n) = \log_2(2^k)$$

$$k = \log_2 n$$

Dengan mengabaikan konstanta basis sesuai aturan perhitungan asimtotik, maka dapat disimpulkan dan dibuktikan bahwa Kompleksitas Waktu *Binary Search* adalah: $\text{Time Complexity} = O(\log n)$ (Sitorus & Hutapea, 2023) (Fitriyani, Achmad H, Lestari, & Fitriawati, 2026).

B. Analisis Kompleksitas Ruang (*Space Complexity*)

Pada implementasi *Binary Search* versi iteratif, program tidak melakukan duplikasi himpunan array masukan ke dalam blok memori baru. Sistem murni hanya memanipulasi variabel primitif (seperti low, high, mid) sebagai penanda alamat indeks. Karena alokasi memori ini bersifat statis dan tidak akan membengkak meskipun jumlah elemen input n mencapai jutaan, maka memori operasional bernilai konstan: $\text{Space Complexity} = O(1)$

6. IMPLEMENTASI DAN PENGUJIAN KODE KELUARAN

Berikut merupakan perancangan kode program menggunakan bahasa Python yang mencerminkan alur logika *Binary Search*:

```
def eksekusi_binary_search(array_terurut, nilai_target):
    # Inisialisasi batas awal pencarian
    low = 0
    high = len(array_terurut) - 1

    # Looping berjalan selama batas aktif valid
    while low <= high:
        # Menghitung titik tengah
        mid = (low + high) // 2

        # Skenario 1: Target ditemukan
        if array_terurut[mid] == nilai_target:
            return mid

        # Skenario 2: Target > nilai tengah (eliminasi sisi kiri)
        elif array_terurut[mid] < nilai_target:
            low = mid + 1

        # Skenario 3: Target < nilai tengah (eliminasi sisi kanan)
        else:
            high = mid - 1

    # Mengembalikan nilai -1 jika target tidak ada dalam array
    return -1

# ----- BLOK PENGUJIAN STUDI KASUS -----
dataset = [5, 12, 18, 23, 35, 44, 57, 62, 78, 89]
target_dicari = 57

print("Memulai Pencarian Biner...")
hasil_indeks = eksekusi_binary_search(dataset, target_dicari)

if hasil_indeks != -1:
    print(f"Target {target_dicari} sukses ditemukan pada indeks ke-
{hasil_indeks}.")
else:
    print("Target tidak dapat ditemukan di dalam dataset.")
```

Gambar 1. Implementasi Algoritma Binary Search menggunakan Bahasa Python

Berdasarkan Gambar 1 di atas, fungsi utama dari kode tersebut adalah mengeksekusi pencarian secara logaritmik dengan menerima dua parameter input, yaitu himpunan data yang sudah terurut (*array_terurut*) dan nilai yang ingin dicari (*nilai_target*). Program bekerja dengan memanfaatkan perulangan *while* yang akan terus berjalan selama batas bawah (*low*) tidak melebihi batas atas (*high*). Di dalam perulangan tersebut, program menghitung titik tengah (*mid*) dan melakukan tiga kondisi evaluasi. Jika elemen di posisi *mid* bernilai sama dengan target, maka pencarian selesai dan program mengembalikan

posisi indeksnya. Namun, jika target lebih besar atau lebih kecil dari nilai tengah, program akan otomatis membuang setengah bagian array dengan menggeser variabel low atau high. Jika perulangan berakhir dan target tetap tidak ditemukan, fungsi akan mengembalikan nilai -1 sebagai indikator bahwa data tidak terdapat di dalam sistem.

7. KESIMPULAN

Berdasarkan pengujian studi kasus dan analisis asimtotik yang telah dilakukan, dapat disimpulkan bahwa algoritma *Binary Search* merupakan instrumen pencarian yang sangat efisien dan superior untuk menangani himpunan data berskala masif, dengan syarat mutlak data tersebut telah terurut. Melalui penerapan paradigma *Divide and Conquer*, algoritma ini mampu mengeliminasi setengah dari ruang pencarian pada setiap iterasinya. Hal ini terbukti dari simulasi pencarian angka yang hanya membutuhkan 4 kali iterasi evaluasi, dibandingkan dengan *Linear Search* yang membutuhkan hingga 7 kali iterasi untuk kasus yang sama. Pembuktian matematis menunjukkan bahwa pemotongan rentang data secara konsisten ini mereduksi kompleksitas waktu secara drastis dari tingkat linear menjadi logaritmik, yaitu $O(\log n)$. Di samping itu, implementasi iteratif algoritma ini murni hanya memanipulasi penunjuk indeks tanpa menduplikasi data, sehingga sangat hemat memori dengan kompleksitas ruang bernilai konstan $O(1)$. Hal ini menjadi bukti nyata bahwa optimalisasi desain algoritma memiliki dampak efisiensi performa komputasi yang jauh lebih signifikan ketimbang semata-mata mengandalkan peningkatan kapasitas perangkat keras (*hardware*).

8. DAFTAR PUSTAKA

- Aviantika, R. D., Kustanto, K., & Hasbi, M. (2021). Pencarian Data Barang Produk Atribut Sekolah Menggunakan Algoritma Binary Search. *Jurnal Restikom*, 101-108.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms (4th Edition)*. MIT Press.
- Fitriyani, A., Achmad H, A., Lestari, D., & Fitriawati, N. (2026). Penerapan Algoritma Binary Search Pencarian Data Pada Sistem Supply Barang. *JSI (Jurnal Sistem Informasi) Universitas Suryadarma*, 112-120.
- Haming, N., Lestanti, S., & Budiman, S. N. (2021). Aplikasi Informasi Spesialite Obat Indonesia Berbasis Web Menggunakan Metode Pencarian Binary Search. *DECODE: Jurnal Pendidikan Teknologi Informasi*, 55-63.
- Putri, A. M., & Wibisono, I. S. (2022). Penerapan Binary Search Pada Aplikasi Penjualan Berbasis Web Studi Kasus Pada Toko More Shop. *Jurnal Multimatrix*, 15-23.
- Ramadhan, H., Fauziah, F., & Lantana, D. A. (2023). Perbandingan Algoritma Binary Search dan Sequential Search untuk Pencarian Persediaan Stok Barang Berbasis Web. *STRING (Satuan Tulisan Riset dan Inovasi Teknologi)*, 45-54.
- Religia, Y., Rohim, M. F., & Hapsari, R. (2019). Analisis Algoritma Sequential Search Dan Binary Search Pada Big Data. *Jurnal Ilmiah Informatika, Arsitektur Dan Lingkungan*, 77-85.

- Sitorus, J., & Hutapea, R. (2023). *Analisis Komparatif Efisiensi Waktu Algoritma Linear Search dan Binary Search pada Array Terurut*. Jurnal Sains dan Teknologi Komputer (Format Buku/Prosiding).
- Widodoputro, L., Harsani, P., & Mulyana, I. (2018). Penerapan Metode Binary Search (Pencarian Biner) pada Buku Resep Masakan Berbasis Android Mobile. *Pustaka: Jurnal Studi Perpustakaan dan Informasi*, 32-40.